University of Asia Pacific

Department of Computer Science and Engineering

CSE 402 : Operating System Lab

LAB REPORT

Lab Report Number: 06

Experiment : Implementation of Semaphore for Process Synchronization in C using Linux

Submitted by:

Name : H. M. Tahsin Sheikh

Student ID : 22201243

Section : E1

Submitted to:

Bidita Sarkar Diba
Lecturer,
Department of Computer Science and Engineering

Date of Submission: 20/05/2026

1. Title of the Experiment

Implementation of Semaphore for Process Synchronization in C using Linux

2. Objective

- To understand the concept of semaphore in Operating System.
- To learn process synchronization using semaphores.
- To create multiple processes using fork().
- To control concurrent access to the critical section using sem_wait() and sem_post().

3. Introduction

A semaphore is a synchronization mechanism used in Operating Systems to control access to shared resources by multiple processes or threads. It helps prevent race conditions and ensures proper execution order.

In this experiment, the semaphore variable mutex is used to protect the critical section where processes print messages to the screen.

The following semaphore functions are used:

- sem_init() → Initializes the semaphore.
- sem_wait() → Decreases semaphore value and enters critical section.
- sem_post() → Increases semaphore value and exits critical section.

The program creates multiple child processes using fork(). Since multiple processes run concurrently, semaphore ensures that only one process accesses the critical section at a time.

4. Algorithm

1. Start the program.
2. Initialize semaphore mutex with value 1.
3. Create first child process using fork().
4. Create second child process using another fork().

5. Check process conditions:
    - If rc1 == 0, execute first child process.
    - Else if rc2 == 0, execute second child process.
    - Otherwise, execute parent process.
6. Each process executes a loop two times.
7. Before printing:
    - Enter critical section using sem_wait().
8. Print process ID.
9. Exit critical section using sem_post().
10. End program.

5. Source Code

```
#include <stdio.h>

#include <stdlib.h>

#include <unistd.h>

#include <sys/wait.h>

#include <semaphore.h>


sem_t mutex;

int main(int argc, char *argv[]) {

    sem_init(&mutex, 1, 1);

    int rc1 = fork();

    int rc2 = fork();

    if (rc1 == 0) {

        // First child process

        printf("Child Process (pid = %d, parent pid = %d)\n",

                getpid(), getppid());
```

```
    } else if (rc2 == 0) {
        // Second child process
        printf("Child Process (pid = %d, parent pid = %d)\n",
                getpid(), getppid());
    } else {
        // Parent process
        printf("Parent Process (pid = %d)\n", getpid());
    }
    // Each process prints twice
    for (int i = 0; i < 2; i++) {
        sem_wait(&mutex);
        printf("Hello from (pid = %d)\n", (int)getpid());
        sem_post(&mutex);
    }
    return 0;
}
```

6. Input/Output

Sample Output -

```
Parent Process (pid = 2100)

Child Process (pid = 2101, parent pid = 2100)

Child Process (pid = 2102, parent pid = 2100)

Hello from (pid = 2100)
Hello from (pid = 2100)

Hello from (pid = 2101)
```

Hello from (pid = 2101)

Hello from (pid = 2102)
Hello from (pid = 2102)

7. Conclusion

From this experiment, we learned how semaphores are used for process synchronization in Operating Systems. The semaphore successfully controlled access to the critical section and prevented multiple processes from executing the printing statement simultaneously. This experiment also demonstrated process creation using **fork**() and synchronization using **sem_wait**() and **sem_post**().